

Composite

Composite este un pattern de proiectare structural care permite compunerea obiectelor într-o structură de tip arbore (tree structure) pentru a reprezenta relații de tip parte-întreg.

Codul sursă:

```
// Component
abstract class Component
{
    public abstract void Operation();

    public virtual void Add(Component component)
    {
    }

    public virtual void Remove(Component component)
    {
    }

    public virtual Component GetChild(int index)
    {
        return null;
    }
}

// Leaf
class Leaf : Component
{
    public override void Operation()
    {
        Console.WriteLine("Leaf operation");
    }
}

// Composite
class Composite : Component
{
    private List<Component> children = new List<Component>();

    public override void Add(Component component)
    {
        children.Add(component);
    }

    public override void Remove(Component component)
    {
        children.Remove(component);
    }

    public override void Operation()
    {
        foreach (Component component in children)
        {
            component.Operation();
        }
    }
}

// Client
void Main()
{
    Composite root = new Composite();
    root.Add(new Leaf());
    root.Add(new Leaf());

    root.Operation();
}
```

Implementarea codului sursă:

```
/// Component - defineste interfata comuna pentru produse simple si pachete
public abstract class CatalogItem
{
    public string Name { get; set; }

    protected CatalogItem(string name)
    {
        Name = name;
    }

    public abstract decimal GetPrice();
    public abstract void Display(int depth);
}

/// Leaf - produs simplu
public class SimpleProduct : CatalogItem
{
    private decimal _price;

    public SimpleProduct(string name, decimal price) : base(name)
    {
        _price = price;
    }

    public override decimal GetPrice()
    {
        return _price;
    }

    public override void Display(int depth)
    {
        Console.WriteLine(new string(' ', depth) + $"- {Name} : {_price} MDL");
    }
}

/// Composite - pachet de produse
public class ProductBundle : CatalogItem
{
    private List<CatalogItem> _items = new List<CatalogItem>();

    public ProductBundle(string name) : base(name)
    {
    }

    public void Add(CatalogItem item)
    {
        _items.Add(item);
    }

    public void Remove(CatalogItem item)
    {
        _items.Remove(item);
    }

    public override decimal GetPrice()
    {
        decimal total = 0;
        foreach (var item in _items)
        {
            total += item.GetPrice();
        }
        return total;
    }

    public override void Display(int depth)
    {
        Console.WriteLine(new string(' ', depth) + $"+ {Name} (Total: {GetPrice()} MDL)");

        foreach (var item in _items)
        {
            item.Display(depth + 2);
        }
    }
}
```

Main:

```
void Main()
{
    // Produse simple
    CatalogItem laptop = new SimpleProduct("Laptop", 15000);
    CatalogItem mouse = new SimpleProduct("Mouse", 400);
    CatalogItem keyboard = new SimpleProduct("Keyboard", 700);
    CatalogItem headset = new SimpleProduct("Headset", 900);
    CatalogItem webcam = new SimpleProduct("Webcam", 1200);

    // Pachet simplu
    ProductBundle officeKit = new ProductBundle("Office Kit");
    officeKit.Add(mouse);
    officeKit.Add(keyboard);

    // Alt pachet
    ProductBundle streamingKit = new ProductBundle("Streaming Kit");
    streamingKit.Add(webcam);
    streamingKit.Add(headset);

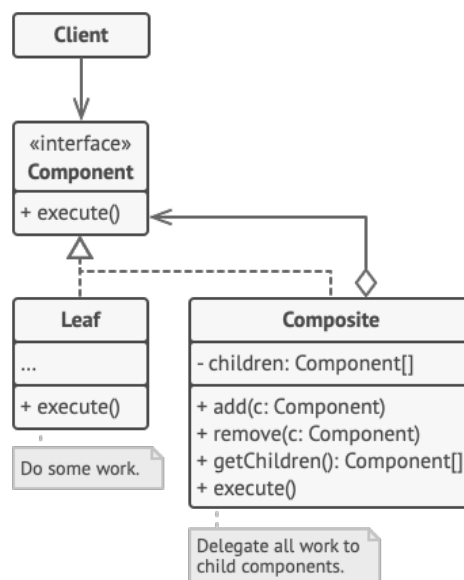
    // Pachet complex care contine produse simple + alte pachete
    ProductBundle laptopBundle = new ProductBundle("Laptop Premium Bundle");
    laptopBundle.Add(laptop);
    laptopBundle.Add(officeKit);
    laptopBundle.Add(streamingKit);

    // Afisare structurata
    laptopBundle.Display(0);

    Console.WriteLine();
    Console.WriteLine("Pret total final: " + laptopBundle.GetPrice() + " MDL");

    Console.ReadKey();
}
```

Diagrama UML:



Domenii de aplicare:

Pattern-ul Composite se utilizează în următoarele situații:

- Când sistemul trebuie să modeleze structuri ierarhice de tip arbore (tree structure), unde obiectele pot conține alte obiecte de același tip.
- Când există relații de tip parte-întreg, iar elementele simple și grupurile de elemente trebuie tratate în mod uniform.
- Când dorim ca aplicația să poată lucra cu obiecte individuale și cu grupuri de obiecte folosind aceeași interfață, fără a face distincție între ele.

- Când dorim construirea unor structuri recursive de obiecte, în care containerele pot include atât elemente simple, cât și alte containere.
- Când dorim simplificarea codului client, astfel încât acesta să nu fie nevoit să verifice tipul concret al obiectelor cu care interacționează.

Avantaje	Dezavantaje
- Permite lucrul mai eficient cu structuri ierarhice complexe, utilizând polimorfismul și recursivitatea pentru manipularea obiectelor. - Permite tratarea uniformă a obiectelor simple și a grupurilor de obiecte, simplificând codul client. - Respectă principiul Open/Closed Principle, deoarece pot fi adăugate noi tipuri de componente fără a modifica codul existent. - Simplifică manipularea structurilor de tip arbore prin utilizarea unei interfețe comune.	- Poate fi dificilă definirea unei interfețe comune pentru clase ale căror funcționalități diferă semnificativ. - Interfața componentelor poate deveni prea generală pentru a acoperi toate cazurile posibile. - În anumite situații, implementarea poate deveni mai dificil de înțeles din cauza structurii recursive.